

Flutter & Dart Complete Learning Roadmap

From Dart Fundamentals to Capstone Projects

A Complete Study Guide

2026

Introduction

PART 1: DART FUNDAMENTALS

Phase 1 - Dart Fundamentals

Phase 2 - Object-Oriented Programming

Phase 3 - Advanced Dart

PART 2: FLUTTER

Flutter Basics

Layout Widgets

Basic Widgets

Input Widgets

Navigation

State Management

Networking

Local Database

Firebase

Responsive UI

Animations

Device Features

Architecture

Testing

Deployment

Git

Projects (Practice)

Final Notes

Introduction

This guide is a complete, structured roadmap for learning **Dart** and **Flutter** — from absolute basics to building real-world, production-ready mobile applications. Every topic includes a plain-language explanation of *what it is*, *why it matters*, and a working **code example** you can type out and run yourself.

Work through the phases in order. Dart fundamentals and OOP come first because Flutter is just Dart + a widget framework — if the Dart underneath is shaky, Flutter will feel confusing. Once the Dart section is solid, the Flutter widget system will click much faster.

PART 1: DART FUNDAMENTALS

Phase 1 - Dart Fundamentals

► 1. Install Flutter SDK & IDE (VS Code / Android Studio)

Explanation: Flutter is Google's UI toolkit for building natively compiled apps for mobile, web, and desktop from a single codebase. Dart is the programming language Flutter uses. Before writing any code you need:

1. The **Flutter SDK** (download from flutter.dev or clone via Git) added to your system PATH.
2. An **IDE** — VS Code (lightweight, with the Flutter + Dart extensions) or Android Studio (heavier, built-in emulator management).
3. Run `flutter doctor` in your terminal — it checks your setup and tells you exactly what's missing (Android SDK, Xcode, licenses, etc.).

Example (terminal commands):

```
# Check installation and environment issues
flutter doctor

# Create a new project
flutter create my_first_app

# Run the app on a connected device/emulator
cd my_first_app
flutter run
```

► 2. Understand Flutter Project Structure

Explanation: A generated Flutter project has a predictable folder layout. Knowing what each folder does saves hours of confusion later.

```
my_first_app/
├── android/      # Native Android project (Gradle, manifests)
├── ios/          # Native iOS project (Xcode config)
├── lib/          # YOUR Dart code lives here
│   └── main.dart # Entry point of the app
├── test/         # Unit/widget tests
├── assets/       # Images, fonts (you create this)
├── pubspec.yaml  # Project config: dependencies, assets, metadata
└── pubspec.lock  # Locked/resolved dependency versions
```

- `pubspec.yaml` is like `package.json` in Node.js — it declares dependencies and assets.
- `lib/main.dart` must contain a `main()` function — this is where execution starts.
- `android/` and `ios/` are only touched when you need native-level configuration (permissions, app icons, signing).

Example `main.dart`:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(child: Text('Hello, Flutter!')),
      ),
    );
  }
}
```

► 3. Variables (`var`, `final`, `const`)

Explanation: - `var` — type is inferred once at assignment; the value can change later. - `final` — the variable can be set **only once**, but its value is determined **at runtime**. - `const` — the value must be known **at compile time** and is deeply immutable.

Example:

```
void main() {
  var name = "Abigail";      // inferred as String, can be reassigned
  name = "Abby";            // OK

  final DateTime now = DateTime.now(); // set once, known only at runtime
  // now = DateTime.now();           // ERROR: can't reassign a final variable

  const double pi = 3.14159; // compile-time constant
  const List<int> scores = [98, 85, 100]; // deeply immutable list

  print("$name logged in at $now. Pi is $pi. Scores: $scores");
}
```

► 4. Data Types

Explanation: Dart is statically typed but supports type inference. Core built-in types:

Type	Example	Notes
<code>int</code>	<code>int age = 21;</code>	Whole numbers
<code>double</code>	<code>double gpa = 3.75;</code>	Decimal numbers
<code>num</code>	<code>num x = 5;</code>	Parent of int/double
<code>String</code>	<code>String name = "Abby";</code>	Text
<code>bool</code>	<code>bool isActive = true;</code>	true/false
<code>List</code>	<code>List<int> nums = [1,2,3];</code>	Ordered collection
<code>Set</code>	<code>Set<String> tags = {"a","b"};</code>	Unique unordered items
<code>Map</code>	<code>Map<String,int> ages = {"Abby":21};</code>	Key-value pairs
<code>dynamic</code>	<code>dynamic x = "hi";</code>	Type checked at runtime only

Example:

```
void main() {
  int age = 21;
  double gpa = 3.75;
  String course = "BS Computer Science";
  bool isEnrolled = true;
  dynamic anything = "could be any type";

  print("$course student, age $age, gpa $gpa, enrolled: $isEnrolled");
  anything = 42; // allowed because it's dynamic
}
```

► 5. Operators

Explanation: Dart supports arithmetic, relational, logical, assignment, and special operators like `??` (null-coalescing) and `?.` (null-aware access).

Example:

```
void main() {
  int a = 10, b = 3;

  print(a + b); // Arithmetic: 13
  print(a % b); // Modulo: 1
  print(a > b); // Relational: true
  print(a > 5 && b < 5); // Logical AND: true

  int? score; // nullable int
  int finalScore = score ?? 0; // null-coalescing: use 0 if score is null
  print(finalScore); // 0

  String? maybeName;
  print(maybeName?.toUpperCase()); // null-aware call, prints "null" safely
}
```

► 6. Input and Output

Explanation: Output uses `print()`. Console input (in pure Dart, non-Flutter) uses `dart:io`'s `stdin.readLineSync()`. In Flutter apps, "input" instead comes from widgets like `TextField` (covered later) since Flutter apps don't have a terminal for the end user.

Example (pure Dart console app):

```
import 'dart:io';

void main() {
  stdout.write("Enter your name: ");
  String? name = stdin.readLineSync();
  print("Hello, $name! Welcome to Dart.");
}
```

► 7. If-else

Explanation: Standard conditional branching. Dart also supports the ternary operator `condition ? a : b` for short conditionals.

Example:

```
void main() {
  int grade = 85;

  if (grade >= 90) {
    print("Excellent");
  } else if (grade >= 75) {
    print("Passed");
  } else {
    print("Failed");
  }

  // Ternary shorthand
  String result = grade >= 75 ? "Passed" : "Failed";
  print(result);
}
```

► 8. Switch

Explanation: `switch` compares a value against multiple constant cases — cleaner than long if-else chains. Modern Dart (3.x) also supports switch **expressions** and pattern matching.

Example:

```
void main() {
  String role = "admin";

  switch (role) {
    case "admin":
      print("Full access granted");
      break;
    case "editor":
      print("Can edit content");
      break;
    default:
      print("Read-only access");
  }

  // Dart 3 switch expression
  String message = switch (role) {
    "admin" => "Full access granted",
    "editor" => "Can edit content",
    _ => "Read-only access",
  };
  print(message);
}
```

► 9. Loops (for, while, do-while)

Explanation: - `for` — when you know how many times to repeat, or are iterating a collection. - `while` — repeats while a condition stays true (may run zero times). - `do-while` — same as while, but runs **at least once**.

Example:

```

void main() {
  // for loop
  for (int i = 1; i <= 3; i++) {
    print("for loop: $i");
  }

  // for-in (iterating a collection)
  for (var fruit in ["apple", "banana"]) {
    print("fruit: $fruit");
  }

  // while loop
  int count = 0;
  while (count < 3) {
    print("while loop: $count");
    count++;
  }

  // do-while loop
  int x = 0;
  do {
    print("do-while: $x");
    x++;
  } while (x < 3);
}

```

► 10. Functions

Explanation: Functions are first-class citizens in Dart — they can be assigned to variables, passed as arguments, and returned from other functions.

Example:

```

// Basic function with typed parameter and return type
int add(int a, int b) {
  return a + b;
}

// Function with default parameter
void greet(String name, {String greeting = "Hello"}) {
  print("$greeting, $name!");
}

void main() {
  print(add(3, 4)); // 7
  greet("Abigail"); // Hello, Abigail!

  // Function assigned to a variable
  int Function(int, int) multiply = (a, b) => a * b;
  print(multiply(3, 4)); // 12
}

```

► 11. Arrow Functions

Explanation: The `=>` (fat arrow) syntax is shorthand for a function body containing a single expression. Very common in Flutter for short callbacks.

Example:

```

int square(int x) => x * x;

void main() {
  print(square(5)); // 25

  // Common in Flutter: onPressed: () => print("Tapped!")
  var greetUser = (String name) => "Hi, $name!";
  print(greetUser("Abby"));
}

```

► 12. Optional & Named Parameters

Explanation: - **Positional optional** parameters use `[]` — passed in order, may be omitted. - **Named** parameters use `{}` — passed by name, order doesn't matter. Mark with `required` to force the caller to supply them.

Example:

```
// Positional optional
String buildName(String first, [String? last]) {
  return last == null ? first : "$first $last";
}

// Named parameters (very common in Flutter widget constructors)
void createUser({required String name, int age = 18, String? email}) {
  print("Name: $name, Age: $age, Email: ${email ?? 'N/A'}");
}

void main() {
  print(buildName("Abegail")); // Abegail
  print(buildName("Abegail", "Trazona")); // Abegail Trazona

  createUser(name: "Napone", age: 20);
}
```

► 13. Null Safety

Explanation: Since Dart 2.12, types are **non-nullable by default**. To allow `null`, append `?` to the type. This prevents an entire category of runtime crashes (“null reference exceptions”) by catching them at compile time.

Key operators: - `?` — declares a nullable type - `??` — provide a fallback if null - `??=` — assign only if currently null - `!` — “trust me” operator, asserts a value is not null (use carefully)

Example:

```
void main() {
  String? nickname; // nullable, defaults to null
  print(nickname ?? "No nickname set"); // fallback

  nickname ??= "Abby"; // assign only if null
  print(nickname);

  int? age = 21;
  int safeAge = age!; // asserts non-null — throws if age were actually null
  print(safeAge);
}
```

► 14. Collections (List, Set, Map)

Explanation: - **List** — ordered, allows duplicates. Like an array. - **Set** — unordered, unique values only. - **Map** — key-value pairs, keys are unique.

Example:

```
void main() {
  // List
  List<String> fruits = ["apple", "banana", "apple"];
  fruits.add("mango");
  print(fruits); // [apple, banana, apple, mango]

  // Set (duplicates automatically removed)
  Set<String> uniqueFruits = {"apple", "banana", "apple"};
  print(uniqueFruits); // {apple, banana}

  // Map
  Map<String, int> studentGrades = {
    "Abegail": 95,
    "Napone": 88,
  };
  studentGrades["Trazona"] = 90;
  print(studentGrades["Abegail"]); // 95

  // Useful collection methods
  var doubled = fruits.map((f) => f.toUpperCase()).toList();
  var onlyA = fruits.where((f) => f.startsWith("a")).toList();
  print(doubled);
  print(onlyA);
}
```

► 15. Enums

Explanation: Enums define a fixed set of named constant values — useful for representing states, roles, or categories with type safety (instead of using raw strings). Dart 2.17+ supports **enhanced enums** with fields and methods.

Example:

```
enum Status { pending, approved, rejected }

// Enhanced enum with a field and method
enum LaneRole {
  gold(damage: "physical"),
  exp(damage: "physical"),
  mid(damage: "magic");

  final String damage;
  const LaneRole({required this.damage});
}

void main() {
  Status current = Status.approved;

  if (current == Status.approved) {
    print("Request approved!");
  }

  print(LaneRole.gold.damage); // physical
}
```

► 16. Exception Handling

Explanation: Use `try / catch / finally` to gracefully handle runtime errors instead of crashing the app. `throw` raises a custom exception.

Example:

```
void divide(int a, int b) {
  if (b == 0) {
    throw ArgumentError("Cannot divide by zero");
  }
  print(a / b);
}

void main() {
  try {
    divide(10, 0);
  } on ArgumentError catch (e) {
    print("Error caught: ${e.message}");
  } catch (e) {
    print("Unexpected error: $e");
  } finally {
    print("Operation attempted.");
  }
}
```

► 17. Libraries and Imports

Explanation: Dart code is organized into libraries. `import` brings in Dart's core libraries (`dart:core`, `dart:math`, `dart:io`), external packages from pub.dev, or your own project files.

Example:

```
import 'dart:math'; // Dart core library
import 'package:flutter/material.dart'; // external package
// import 'utils/helpers.dart'; // your own file (relative import)

void main() {
  print(sqrt(16)); // 4.0, from dart:math
}
```

Phase 2 - Object-Oriented Programming

► 1. Classes & Objects

Explanation: A **class** is a blueprint; an **object** is an instance created from that blueprint. Classes bundle related data (fields) and behavior (methods) together.

Example:

```
class Student {
  String name;
  int age;

  Student(this.name, this.age); // constructor

  void introduce() {
    print("Hi, I'm $name, $age years old.");
  }
}

void main() {
  Student s1 = Student("Abigail", 21); // object
  s1.introduce();
}
```

► 2. Constructors

Explanation: A constructor initializes an object's fields when it's created. Dart offers a shorthand: `this.fieldName` in the parameter list.

Example:

```
class Book {
  String title;
  double price;

  // Shorthand constructor
  Book(this.title, this.price);
}

void main() {
  Book b = Book("Clean Architecture", 299.0);
  print("${b.title} - P${b.price}");
}
```

► 3. Named Constructors

Explanation: Dart allows multiple constructors per class by giving them names — useful for different ways of building the same type of object.

Example:

```
class Point {
  double x, y;

  Point(this.x, this.y);

  // Named constructor
  Point.origin() : x = 0, y = 0;

  Point.fromMap(Map<String, double> map)
    : x = map["x"] ?? 0,
      y = map["y"] ?? 0;
}

void main() {
  Point p1 = Point(1, 4);
  Point p2 = Point.origin();
  Point p3 = Point.fromMap({"x": 5, "y": 10});

  print("${p1.x},${p1.y}  ${p2.x},${p2.y}  ${p3.x},${p3.y}");
}
```

► 4. Getters & Setters

Explanation: Getters/setters let you control read/write access to a field, e.g. to validate data or compute derived values on the fly, while still using property-style syntax (no parentheses).

Example:


```

class Rectangle {
  double width, height;
  Rectangle(this.width, this.height);

  double get area => width * height; // getter

  set width (double value) { // (avoiding name clash for demo)
    if (value < 0) throw ArgumentError("Width can't be negative");
    width = value;
  }
}

void main() {
  Rectangle r = Rectangle(4, 5);
  print(r.area); // 20.0 - called like a field, no ()
}

```

► 5. Encapsulation

Explanation: Encapsulation hides internal implementation details and exposes only what's necessary. In Dart, prefixing a field/method name with an underscore `_` makes it **private to its library (file)**.

Example:

```

class BankAccount {
  double _balance = 0; // private field

  void deposit(double amount) {
    if (amount > 0) _balance += amount;
  }

  double get balance => _balance; // controlled read access
}

void main() {
  BankAccount acc = BankAccount();
  acc.deposit(500);
  print(acc.balance); // 500.0
  // acc._balance is not accessible outside this file
}

```

► 6. Inheritance

Explanation: A class can **extend** another class to reuse and specialize its fields/methods. The child class is a “subclass”; the parent is the “superclass.”

Example:

```

class Animal {
  String name;
  Animal(this.name);

  void makeSound() => print("$name makes a sound");
}

class Dog extends Animal {
  Dog(super.name);

  @override
  void makeSound() => print("$name barks");
}

void main() {
  Dog d = Dog("Rex");
  d.makeSound(); // Rex barks
}

```

► 7. Polymorphism

Explanation: Polymorphism means objects of different classes can be treated through a common parent type, while each responds to the same method call in its own way.

Example:

```

class Shape {
  double area() => 0;
}

class Circle extends Shape {
  double radius;
  Circle(this.radius);
  @override
  double area() => 3.14159 * radius * radius;
}

class Square extends Shape {
  double side;
  Square(this.side);
  @override
  double area() => side * side;
}

void main() {
  List<Shape> shapes = [Circle(1), Square(4)];
  for (var shape in shapes) {
    print(shape.area().toStringAsFixed(2)); // each computes differently
  }
}

```

► 8. Abstraction

Explanation: An `abstract` class defines a contract (methods without full implementation) that subclasses must fulfill. Abstract classes cannot be instantiated directly.

Example:

```

abstract class PaymentMethod {
  void pay(double amount); // no body – must be implemented by subclasses
}

class GCashPayment extends PaymentMethod {
  @override
  void pay(double amount) => print("Paid P$amount via GCash");
}

void main() {
  PaymentMethod payment = GCashPayment();
  payment.pay(250.0);
  // PaymentMethod p = PaymentMethod(); // ERROR: cannot instantiate abstract class
}

```

► 9. Interfaces

Explanation: Dart has no separate `interface` keyword — **every class implicitly defines an interface**. Use `implements` to force a class to provide its own implementation of all members of another class/interface.

Example:

```

class Printable {
  void printData() {}
}

class Invoice implements Printable {
  @override
  void printData() => print("Printing invoice...");
}

void main() {
  Invoice inv = Invoice();
  inv.printData();
}

```

► 10. Mixins

Explanation: A `mixin` lets you share behavior across unrelated classes without using inheritance (Dart doesn't support multiple inheritance). Apply with `with`.

Example:

```

mixin Logger {
  void log(String message) => print("[LOG]: $message");
}

class OrderService with Logger {
  void placeOrder() {
    log("Order placed successfully");
  }
}

void main() {
  OrderService().placeOrder();
}

```

► 11. Extension Methods

Explanation: Extensions add new functionality to **existing classes** (even ones you don't own, like `String` or `int`) without modifying their source code.

Example:

```

extension StringCasingExtension on String {
  String capitalize() {
    if (isEmpty) return this;
    return "${this[0].toUpperCase()}${substring(1)}";
  }
}

void main() {
  print("abigail".capitalize()); // Abigail
}

```

► 12. Generics

Explanation: Generics let a class/function work with **any type** while still preserving type safety — you write the logic once instead of duplicating it per type.

Example:

```

class Box<T> {
  T value;
  Box(this.value);

  void show() => print("Box contains: $value");
}

T getFirst<T>(List<T> items) => items.first;

void main() {
  Box<int> intBox = Box<int>(12);
  Box<String> strBox = Box<String>("Hello");
  intBox.show();
  strBox.show();

  print(getFirst<String>(["a", "b", "c"])); // a
}

```

Phase 3 - Advanced Dart

► 1. Futures

Explanation: A `Future` represents a value that will be available **later** — the result of an asynchronous operation like a network call or file read. It can complete with a value or an error.

Example:

```

Future<String> fetchUserName() {
  return Future.delayed(const Duration(seconds: 2), () => "Abigail");
}

void main() {
  print("Fetching...");
  fetchUserName().then((name) {
    print("Fetched: $name");
  });
}

```

► 2. Async & Await

Explanation: `async` / `await` is syntactic sugar over Futures — it lets asynchronous code read top-to-bottom like synchronous code, instead of chaining `.then()` calls.

Example:

```
Future<String> fetchData() async {
  await Future.delayed(const Duration(seconds: 1));
  return "Data loaded";
}

void main() async {
  print("Start");
  String result = await fetchData(); // pauses here until the Future completes
  print(result);
  print("End");
}
```

► 3. Streams

Explanation: A `Stream` is like a Future but delivers **multiple values over time** (e.g. real-time data, user input events, or Firestore document changes). Listen with `.listen()` or consume with `await for`.

Example:

```
Stream<int> countStream() async* {
  for (int i = 1; i <= 3; i++) {
    await Future.delayed(const Duration(seconds: 1));
    yield i;
  }
}

void main() async {
  await for (var value in countStream()) {
    print("Received: $value");
  }
  print("Stream done");
}
```

► 4. Isolates (Basic)

Explanation: Dart is single-threaded by default. **Isolates** are separate memory/execution threads used for CPU-heavy work (e.g. parsing huge JSON, image processing) so the UI thread doesn't freeze. They don't share memory — communication happens via message passing.

Example:

```
import 'dart:isolate';

void heavyTask(SendPort sendPort) {
  int sum = 0;
  for (int i = 0; i < 1000000000; i++) {
    sum += i;
  }
  sendPort.send(sum);
}

void main() async {
  ReceivePort receivePort = ReceivePort();
  await Isolate.spawn(heavyTask, receivePort.sendPort);

  receivePort.listen((message) {
    print("Result from isolate: $message");
    receivePort.close();
  });
}
```

► 5. JSON Serialization

Explanation: Apps constantly convert between Dart objects and JSON (from REST APIs, local storage, etc.). `dart:convert` provides `jsonEncode` / `jsonDecode`. For larger apps, `fromJson` / `toJson` methods (or the `json_serializable` package) keep this organized.

Example:

```
import 'dart:convert';

class User {
  final String name;
  final int age;

  User({required this.name, required this.age});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(name: json['name'], age: json['age']);
  }

  Map<String, dynamic> toJson() => {'name': name, 'age': age};
}

void main() {
  String jsonString = '{"name": "Abigail", "age": 21}';

  Map<String, dynamic> data = jsonDecode(jsonString);
  User user = User.fromJson(data);
  print("${user.name} is ${user.age} years old");

  String encoded = jsonEncode(user.toJson());
  print(encoded);
}
```

► 6. Packages (pub.dev)

Explanation: pub.dev is Dart/Flutter's package repository. Add dependencies to `pubspec.yaml`, then run `flutter pub get` to install them.

Example (`pubspec.yaml`):

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0
  provider: ^6.1.2
```

```
flutter pub get
```

```
import 'package:http/http.dart' as http;

void main() async {
  var response = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/users/1'));
  print(response.body);
}
```

► 7. File Handling (Basic)

Explanation: `dart:io`'s `File` class lets you read/write files on disk — useful for caching data, logs, or exporting reports. (Not available on Flutter Web without special handling.)

Example:

```
import 'dart:io';

void main() async {
  File file = File('notes.txt');

  await file.writeAsString('Thesis meeting at 3PM with Trazona and Napone.');
```

```
String contents = await file.readAsString();
print(contents);
}
```

PART 2: FLUTTER

Flutter Basics

► 1. Widgets

Explanation: In Flutter, **everything is a widget** — text, buttons, padding, even layout and app structure itself. Widgets describe what the UI should look like for a given state, and Flutter re-renders efficiently whenever that state changes.

Example:

```
Widget build(BuildContext context) {  
  return const Text('Widgets all the way down');  
}
```

► 2. StatelessWidget

Explanation: A widget with **no mutable state** — once built, it doesn't change unless its parent rebuilds it with new data. Use for static UI (labels, icons, fixed layouts).

Example:

```
class WelcomeCard extends StatelessWidget {  
  final String name;  
  const WelcomeCard({super.key, required this.name});  
  
  @override  
  Widget build(BuildContext context) {  
    return Text('Welcome, $name!');  
  }  
}
```

► 3. StatefulWidget

Explanation: A widget that **can change over time** — it holds mutable state in a companion `State` object. Call `setState()` to tell Flutter “data changed, please rebuild the UI.”

Example:

```
class Counter extends StatefulWidget {  
  const Counter({super.key});  
  @override  
  State<Counter> createState() => _CounterState();  
}  
  
class _CounterState extends State<Counter> {  
  int _count = 0;  
  
  void _increment() {  
    setState(() {  
      _count++;  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        Text('Count: $_count'),  
        ElevatedButton(onPressed: _increment, child: const Text('Add')),  
      ],  
    );  
  }  
}
```

► 4. Build Method

Explanation: `build()` is called every time Flutter needs to draw (or redraw) a widget. It must be **fast and free of side effects** — it just returns a description of the UI based on current state/props.

Example:

```
@override
Widget build(BuildContext context) {
  // Called on first render AND every time setState() triggers a rebuild
  return Container(color: Colors.blue, child: const Text('Rebuilt each time'));
}
```

► 5. Hot Reload

Explanation: Hot Reload injects updated source code into the running app **without losing state**, so you see UI changes almost instantly (press `r` in terminal, or the lightning icon in your IDE). Hot Restart, by contrast, resets state and reloads from scratch. Hot reload doesn't work for changes to `main()`, native code, or some state-initialization logic.

Example workflow:

```
flutter run
# edit a Text widget's string in your editor, save the file
# press "r" in the terminal → UI updates instantly, counter state preserved
# press "R" → full hot restart, state resets
```

► 6. Widget Tree

Explanation: Flutter UI is a **tree of nested widgets** — each widget can contain child widgets, forming a hierarchy from the root (`MaterialApp`) down to individual `Text` or `Icon` widgets. Flutter uses this tree (plus an internal “Element tree” and “RenderObject tree”) to efficiently diff and repaint only what changed.

Example:

```
MaterialApp(          // root
  home: Scaffold(     // child of MaterialApp
    appBar: AppBar(title: const Text('My App')), // child of Scaffold
    body: Center(      // child of Scaffold
      child: Column(   // child of Center
        children: [
          Text('Hello'), // child of Column
          Icon(Icons.star), // child of Column
        ],
      ),
    ),
  ),
)
```

Layout Widgets

► 1. Scaffold

Explanation: `Scaffold` implements the basic Material Design visual layout — it provides slots for an app bar, body, floating action button, drawer, and bottom navigation bar.

```
Scaffold(
  appBar: AppBar(title: const Text('Home')),
  body: const Center(child: Text('Body content')),
  floatingActionButton: FloatingActionButton(onPressed: () {}, child: const Icon(Icons.add)),
)
```

► 2. AppBar

Explanation: The top navigation bar of a screen — typically holds a title, back button, and action icons.

```
AppBar(
  title: const Text('Dashboard'),
  actions: [IconButton(icon: const Icon(Icons.settings), onPressed: () {})],
)
```

► 3. Container

Explanation: A general-purpose box for styling and positioning a single child — combines padding, margin, decoration (color/border/shadow), and sizing in one widget.

```
Container(
  width: 100,
  height: 80,
  padding: const EdgeInsets.all(10),
  decoration: BoxDecoration(
    color: Colors.green.shade100,
    borderRadius: BorderRadius.circular(10),
  ),
  child: const Text('Styled Box'),
)
```

► 4. SizedBox

Explanation: Forces a child to a specific width/height, or (with no child) acts as fixed empty spacing — cheaper than `Container` when you don't need decoration.

```
Column(
  children: const [
    Text('Above'),
    SizedBox(height: 20), // vertical spacing
    Text('Below'),
  ],
)
```

► 5. Padding

Explanation: Adds empty space around a child widget.

```
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
  child: const Text('Padded text'),
)
```

► 6. Center

Explanation: Centers its child within itself (which expands to fill available space unless constrained).

```
Center(child: const Text('Centered!'))
```

► 7. Align

Explanation: Like `Center`, but lets you position a child at any alignment (e.g. `topRight`, `bottomLeft`) instead of only the center.

```
Align(
  alignment: Alignment.bottomRight,
  child: const Icon(Icons.notifications),
)
```

► 8. Row

Explanation: Lays out children **horizontally**. Use `mainAxisAlignment` / `crossAxisAlignment` to control spacing/alignment.

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: const [Icon(Icons.home), Icon(Icons.search), Icon(Icons.person)],
)
```

► 9. Column

Explanation: Lays out children **vertically** — the vertical counterpart to `Row`.

```
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: const [Text('Title'), Text('Subtitle')],
)
```


► 10. Expanded

Explanation: Forces a child inside a `Row` / `Column` / `Flex` to **fill remaining available space**. Multiple `Expanded` widgets share space proportionally via `flex`.

```
Row(
  children: [
    Expanded(flex: 1, child: Container(color: Colors.red, height: 50)),
    Expanded(flex: 1, child: Container(color: Colors.blue, height: 50)),
  ],
)
```

► 11. Flexible

Explanation: Similar to `Expanded`, but the child can choose to be **smaller** than the allotted space (`fit: FlexFit.loose` is the default), whereas `Expanded` forces it to fill exactly (`FlexFit.tight`).

```
Row(
  children: [
    Flexible(child: Text('This text can shrink to fit', overflow: TextOverflow.ellipsis)),
  ],
)
```

► 12. Stack

Explanation: Overlaps children on top of each other (like layers in Photoshop) — position them precisely with `Positioned`.

```
Stack(
  children: [
    Container(width: 200, height: 200, color: Colors.grey),
    const Positioned(bottom: 10, right: 10, child: Icon(Icons.check_circle, color: Colors.green)),
  ],
)
```

► 13. Wrap

Explanation: Like `Row` / `Column`, but automatically **wraps to the next line** when there's no more space — great for tag lists or chip groups.

```
Wrap(
  spacing: 8,
  runSpacing: 8,
  children: const [Chip(label: Text('Flutter')), Chip(label: Text('Dart')), Chip(label: Text('Firebase'))],
)
```

► 14. SafeArea

Explanation: Pads content to avoid device intrusions — notches, status bars, home indicators — so your UI isn't hidden behind system UI.

```
Scaffold(
  body: SafeArea(
    child: Text('This avoids the notch/status bar'),
  ),
)
```

► 15. ListView

Explanation: A scrollable list of widgets, laid out linearly. `ListView.builder` is the efficient version — it only builds items currently visible on screen (crucial for long lists).

```
ListView.builder(
  itemCount: 50,
  itemBuilder: (context, index) => ListTile(title: Text('Item $index')),
)
```

► 16. GridView

Explanation: Lays out widgets in a **2D scrollable grid** — common for photo galleries, product catalogs, dashboards.

```
GridView.count(  
  crossAxisCount: 2,  
  children: List.generate(6, (i) => Card(child: Center(child: Text('Tile $i')))),  
)
```

► 17. SingleChildScrollView

Explanation: Makes a single, non-list child (e.g. a `Column` of form fields) scrollable when its content might overflow the screen (e.g. keyboard pops up).

```
SingleChildScrollView(  
  child: Column(  
    children: List.generate(20, (i) => Text('Line $i')),  
  ),  
)
```

Basic Widgets

► 1. Text

Explanation: Displays a string of text, with styling for font, size, weight, color.

```
Text(  
  'Hello Flutter',  
  style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold, color: Colors.teal),  
)
```

► 2. RichText

Explanation: Displays text with **multiple styles in one paragraph** (e.g. bold word inside normal text) using `TextSpan` children.

```
RichText(  
  text: const TextSpan(  
    style: TextStyle(color: Colors.black),  
    children: [  
      TextSpan(text: 'Status: '),  
      TextSpan(text: 'Approved', style: TextStyle(fontWeight: FontWeight.bold, color: Colors.green)),  
    ],  
  ),  
)
```

► 3. Image

Explanation: Displays images from assets, network, memory, or files.

```
Image.network('https://picsum.photos/200')  
Image.asset('assets/logo.png')
```

► 4. Icon

Explanation: Displays a glyph from an icon font (Material Icons by default) or custom icon font.

```
Icon(Icons.favorite, color: Colors.red, size: 30)
```

► 5. Card

Explanation: A Material Design surface with rounded corners and elevation (shadow) — commonly wraps list items or grouped content.

```
Card(
  elevation: 4,
  child: Padding(
    padding: const EdgeInsets.all(10),
    child: const Text('Card content'),
  ),
)
```

► 6. CircleAvatar

Explanation: A circular widget typically used for profile pictures or initials.

```
CircleAvatar(
  radius: 30,
  backgroundImage: NetworkImage('https://i.pravatar.cc/150'),
)
```

► 7. Divider

Explanation: A thin horizontal line used to visually separate content sections.

```
Column(children: const [Text('Section A'), Divider(), Text('Section B')])
```

► 8. Chip

Explanation: A compact element representing an attribute, tag, or filter — often used with `Wrap` for tag lists.

```
Chip(
  label: const Text('Gold Lane'),
  avatar: const Icon(Icons.attach_money, size: 18),
)
```

Input Widgets

► 1. TextField

Explanation: A basic text input box. Use a `TextEditingController` to read/set its value programmatically.

```
final controller = TextEditingController();

TextField(
  controller: controller,
  decoration: const InputDecoration(labelText: 'Username', border: OutlineInputBorder()),
)
```

► 2. TextFormField

Explanation: Same as `TextField`, but designed to work inside a `Form` widget with built-in **validation** support.

```
Form(
  key: GlobalKey<FormState>(),
  child: TextFormField(
    decoration: const InputDecoration(labelText: 'Email'),
    validator: (value) {
      if (value == null || !value.contains('@')) return 'Enter a valid email';
      return null;
    },
  ),
)
```

► 3. Buttons

Explanation: Flutter provides several button styles: `ElevatedButton` (filled, raised), `TextButton` (flat, no background), `OutlinedButton` (bordered), and `IconButton` (icon-only, tappable).

```
ElevatedButton(onPressed: () {}, child: const Text('Submit'))
TextButton(onPressed: () {}, child: const Text('Cancel'))
OutlinedButton(onPressed: () {}, child: const Text('Learn More'))
IconButton(icon: const Icon(Icons.delete), onPressed: () {})
```

► 4. Checkbox

Explanation: A binary on/off toggle, typically used in forms/settings lists.

```
bool isChecked = false;
Checkbox(
  value: isChecked,
  onChanged: (value) => setState(() => isChecked = value!),
)
```

► 5. Radio

Explanation: Lets the user choose **one option from a group** of mutually exclusive choices.

```
int? selected = 0;
Column(
  children: [
    RadioListTile(value: 0, groupValue: selected, title: const Text('Option A'), onChanged: (v) => setState(() => selected = v as int?),
    RadioListTile(value: 1, groupValue: selected, title: const Text('Option B'), onChanged: (v) => setState(() => selected = v as int?),
  ],
)
```

► 6. Switch

Explanation: A toggle switch for binary settings (on/off), like a stylized `Checkbox`.

```
bool isDarkMode = false;
Switch(
  value: isDarkMode,
  onChanged: (value) => setState(() => isDarkMode = value),
)
```

► 7. Slider

Explanation: Lets the user select a numeric value from a continuous or stepped range by dragging.

```
double volume = 50;
Slider(
  value: volume,
  min: 0,
  max: 100,
  divisions: 10,
  label: volume.round().toString(),
  onChanged: (value) => setState(() => volume = value),
)
```

► 8. DropdownButton

Explanation: Displays a list of options in a collapsible menu — user picks one value.

```
String? selectedRole = 'Gold Lane';
DropdownButton<String>(
  value: selectedRole,
  items: ['Gold Lane', 'EXP Lane', 'Mid Lane']
    .map((role) => DropdownMenuItem(value: role, child: Text(role)))
    .toList(),
  onChanged: (value) => setState(() => selectedRole = value),
)
```

► 9. DatePicker

Explanation: `showDatePicker` opens a Material calendar dialog for selecting a date.

```
Future<void> pickDate(BuildContext context) async {
  DateTime? picked = await showDatePicker(
    context: context,
    initialDate: DateTime.now(),
    firstDate: DateTime(2020),
    lastDate: DateTime(2030),
  );
  if (picked != null) print('Picked: $picked');
}
```

► 10. TimePicker

Explanation: `showTimePicker` opens a Material time-selection dialog.

```
Future<void> pickTime(BuildContext context) async {
  TimeOfDay? picked = await showTimePicker(
    context: context,
    initialTime: TimeOfDay.now(),
  );
  if (picked != null) print('Picked time: ${picked.format(context)}');
}
```

Navigation

► 1. Navigator

Explanation: `Navigator` manages a stack of screens (`Route`s). `push` adds a new screen on top; `pop` removes the current one and returns to the previous.

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const DetailScreen()),
);

Navigator.pop(context); // go back
```

► 2. Routes

Explanation: A `Route` represents one screen/page. `MaterialPageRoute` gives platform-appropriate transition animations.

```
MaterialPageRoute(builder: (context) => const ProfileScreen())
```

► 3. Named Routes

Explanation: Instead of building routes inline, you register named routes centrally in `MaterialApp` — cleaner for apps with many screens.

```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => const HomeScreen(),
    '/profile': (context) => const ProfileScreen(),
  },
)

// Navigate using the name:
Navigator.pushNamed(context, '/profile');
```

► 4. Passing Data

Explanation: Pass data to the next screen either via constructor arguments (with `MaterialPageRoute`) or as `arguments` with named routes.

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => ProductDetail(productId: 42)),
);

// With named routes:
Navigator.pushNamed(context, '/detail', arguments: 42);
// Then, inside DetailScreen:
final id = ModalRoute.of(context)!.settings.arguments as int;
```

► 5. Returning Data

Explanation: `Navigator.pop(context, result)` sends a value back to the screen that pushed the current one; `await` the `push` call to receive it.

```
// Screen A
final result = await Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const SelectionScreen()),
);
print('User selected: $result');

// Screen B (SelectionScreen)
Navigator.pop(context, 'Option 1');
```

State Management

► 1. setState

Explanation: The simplest way to manage state — built into `StatefulWidget`. Good for small, localized state (a single screen or widget), but doesn't scale well across many widgets/screens.

```
class LikeButton extends StatefulWidget {
  const LikeButton({super.key});
  @override
  State<LikeButton> createState() => _LikeButtonState();
}

class _LikeButtonState extends State<LikeButton> {
  bool liked = false;

  @override
  Widget build(BuildContext context) {
    return IconButton(
      icon: Icon(liked ? Icons.favorite : Icons.favorite_border),
      onPressed: () => setState(() => liked = !liked),
    );
  }
}
```

► 2. Provider

Explanation: `Provider` is a wrapper around Flutter's `InheritedWidget` that makes state accessible to any descendant widget without manually passing it down ("prop drilling"). Widgets that call `context.watch()` rebuild automatically when the data changes.

```
// 1. Define a ChangeNotifier
class CounterModel extends ChangeNotifier {
  int count = 0;
  void increment() {
    count++;
    notifyListeners(); // tells listening widgets to rebuild
  }
}

// 2. Provide it at the top of the widget tree
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CounterModel(),
      child: const MyApp(),
    ),
  );
}

// 3. Consume it anywhere below
class CounterDisplay extends StatelessWidget {
  const CounterDisplay({super.key});
  @override
  Widget build(BuildContext context) {
    final counter = context.watch<CounterModel>();
    return Column(
      children: [
        Text('${counter.count}'),
        ElevatedButton(
          onPressed: () => context.read<CounterModel>().increment(),
          child: const Text('+'),
        ),
      ],
    );
  }
}
```

► 3. Riverpod

Explanation: Riverpod is Provider’s successor by the same author — it fixes limitations like needing `BuildContext` and compile-time safety issues. State is declared as top-level “providers” and can be read from anywhere, including outside the widget tree.

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

// Declare a provider
final counterProvider = StateProvider<int>((ref) => 0);

void main() {
  runApp(const ProviderScope(child: MyApp()));
}

class CounterDisplay extends ConsumerWidget {
  const CounterDisplay({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final count = ref.watch(counterProvider);
    return Column(
      children: [
        Text('$count'),
        ElevatedButton(
          onPressed: () => ref.read(counterProvider.notifier).state++,
          child: const Text('+'),
        ),
      ],
    );
  }
}
```

► 4. Bloc/Cubit (Optional)

Explanation: BLoC (Business Logic Component) strictly separates UI from business logic using **events** (input) and **states** (output). `Cubit` is a simplified BLoC without explicit events — you just call methods that emit new states.

```
import 'package:flutter_bloc/flutter_bloc.dart';

// Cubit: simpler version of Bloc
class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);
  void increment() => emit(state + 1);
}

void main() {
  runApp(
    BlocProvider(
      create: (_) => CounterCubit(),
      child: const MyApp(),
    ),
  );
}

class CounterView extends StatelessWidget {
  const CounterView({super.key});
  @override
  Widget build(BuildContext context) {
    return BlocBuilder<CounterCubit, int>(
      builder: (context, count) => Column(
        children: [
          Text('$count'),
          ElevatedButton(
            onPressed: () => context.read<CounterCubit>().increment(),
            child: const Text('+'),
          ),
        ],
      ),
    );
  }
}
```

Networking

► 1. HTTP Package

Explanation: The `http` package is the standard way to make REST API calls (GET, POST, PUT, DELETE) from Dart/Flutter.

```
dependencies:
  http: ^1.2.0
```

```
import 'package:http/http.dart' as http;

Future<void> fetchExample() async {
  final response = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'));
  print(response.statusCode);
  print(response.body);
}
```

► 2. REST API

Explanation: REST APIs expose resources over HTTP using standard verbs: **GET** (read), **POST** (create), **PUT / PATCH** (update), **DELETE** (remove). Flutter apps consume these to sync with a backend.

```
Future<void> createPost() async {
  final response = await http.post(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'title': 'Farm Forecast', 'body': 'Weather data', 'userId': 1}),
  );
  print('Status: ${response.statusCode}'); // 201 Created
}
```

► 3. JSON Parsing

Explanation: API responses arrive as JSON text — decode with **jsonDecode**, then map into your own Dart model classes for type safety.

```
class Post {
  final int id;
  final String title;
  Post({required this.id, required this.title});

  factory Post.fromJson(Map<String, dynamic> json) =>
    Post(id: json['id'], title: json['title']);
}

Future<Post> fetchPost() async {
  final response = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'));
  return Post.fromJson(jsonDecode(response.body));
}
```

► 4. Error Handling

Explanation: Network calls can fail (no internet, server error, bad status code). Always wrap them in **try/catch** and check **statusCode** before parsing.

```
Future<Post?> safeFetchPost() async {
  try {
    final response = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'));
    if (response.statusCode == 200) {
      return Post.fromJson(jsonDecode(response.body));
    } else {
      print('Server error: ${response.statusCode}');
      return null;
    }
  } catch (e) {
    print('Network error: $e');
    return null;
  }
}
```

► 5. Loading States

Explanation: UI should reflect three states while fetching: **loading**, **success**, and **error** — usually with a **FutureBuilder** or state variables + **setState**/state management.


```
FutureBuilder<Post?>({
  future: safeFetchPost(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const CircularProgressIndicator();
    } else if (snapshot.hasError || snapshot.data == null) {
      return const Text('Failed to load post');
    } else {
      return Text(snapshot.data!.title);
    }
  },
})
```

Local Database

► 1. SharedPreferences

Explanation: Stores small pieces of **key-value** data persistently on-device (settings, flags, tokens). Not for large or structured data.

```
dependencies:
  shared_preferences: ^2.2.3
```

```
import 'package:shared_preferences/shared_preferences.dart';

Future<void> saveUsername(String name) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('username', name);
}

Future<String?> getUsername() async {
  final prefs = await SharedPreferences.getInstance();
  return prefs.getString('username');
}
```

► 2. Hive

Explanation: A fast, lightweight NoSQL key-value database written in pure Dart — no native dependencies, works offline, great for structured objects (like a to-do list or notes app) without needing SQL.

```
dependencies:
  hive: ^2.2.3
  hive_flutter: ^1.1.0
```

```
import 'package:hive_flutter/hive_flutter.dart';

Future<void> main() async {
  await Hive.initFlutter();
  var box = await Hive.openBox('notesBox');

  box.put('note1', 'Buy groceries');
  print(box.get('note1')); // Buy groceries
}
```

► 3. SQLite

Explanation: A full relational (SQL) database engine embedded on-device — best for structured, relational data with complex queries (joins, filters). The **sqflite** package provides Flutter bindings.

```
dependencies:
  sqflite: ^2.3.3
  path: ^1.9.0
```

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

Future<Database> initDb() async {
  return openDatabase(
    join(await getDatabasesPath(), 'app.db'),
    onCreate: (db, version) => db.execute(
      'CREATE TABLE students(id INTEGER PRIMARY KEY, name TEXT, course TEXT)',
    ),
    version: 1,
  );
}

Future<void> insertStudent(Database db, String name, String course) async {
  await db.insert('students', {'name': name, 'course': course});
}

Future<List<Map<String, dynamic>>> getStudents(Database db) async {
  return db.query('students');
}
```

Firebase

► 1. Authentication

Explanation: Firebase Auth provides ready-made sign-up/login flows (email-password, Google, phone, etc.) without building your own auth server.

```
dependencies:
  firebase_auth: ^5.1.0
  firebase_core: ^3.1.0
```

```
import 'package:firebase_auth/firebase_auth.dart';

Future<UserCredential> signUp(String email, String password) {
  return FirebaseAuth.instance.createUserWithEmailAndPassword(
    email: email,
    password: password,
  );
}

Future<UserCredential> signIn(String email, String password) {
  return FirebaseAuth.instance.signInWithEmailAndPassword(
    email: email,
    password: password,
  );
}
```

► 2. Firestore

Explanation: Firestore is Firebase's cloud NoSQL document database — real-time syncing across devices, structured as **collections** of **documents**.

```
dependencies:
  cloud_firestore: ^5.0.1
```

```
import 'package:cloud_firestore/cloud_firestore.dart';

Future<void> addTask(String title) {
  return FirebaseFirestore.instance.collection('tasks').add({
    'title': title,
    'done': false,
    'createdAt': FieldValue.serverTimestamp(),
  });
}

Stream<QuerySnapshot> getTasks() {
  return FirebaseFirestore.instance.collection('tasks').snapshots();
}
```

► 3. Storage

Explanation: Firebase Storage handles uploading/downloading large binary files (images, videos, PDFs) backed by Google Cloud Storage.

```
dependencies:
  firebase_storage: ^12.1.1
```

```
import 'package:firebase_storage/firebase_storage.dart';
import 'dart:io';

Future<String> uploadProfilePicture(File file, String userId) async {
  final ref = FirebaseStorage.instance.ref().child('profile_pics/$userId.jpg');
  await ref.putFile(file);
  return await ref.getDownloadURL();
}
```

► 4. Cloud Messaging

Explanation: Firebase Cloud Messaging (FCM) sends **push notifications** to devices, even when the app is closed.

```
dependencies:
  firebase_messaging: ^15.0.1
```

```
import 'package:firebase_messaging/firebase_messaging.dart';

Future<void> setupPushNotifications() async {
  FirebaseMessaging messaging = FirebaseMessaging.instance;
  await messaging.requestPermission();

  String? token = await messaging.getToken();
  print('Device token: $token');

  FirebaseMessaging.onMessage.listen((RemoteMessage message) {
    print('Notification received: ${message.notification?.title}');
  });
}
```

► 5. Crashlytics

Explanation: Firebase Crashlytics automatically collects crash reports and non-fatal errors from real users in production, so you can diagnose issues without reproducing them yourself.

```
dependencies:
  firebase_crashlytics: ^4.0.1
```

```
import 'package:firebase_crashlytics/firebase_crashlytics.dart';
import 'package:flutter/foundation.dart';

void main() {
  FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError;

  PlatformDispatcher.instance.onError = (error, stack) {
    FirebaseCrashlytics.instance.recordError(error, stack, fatal: true);
    return true;
  };

  runApp(const MyApp());
}
```

Responsive UI

► 1. MediaQuery

Explanation: `MediaQuery` gives you the current screen's size, orientation, padding (notches), and text scale — used to adapt layouts to the device.

```
double screenWidth = MediaQuery.of(context).size.width;
bool isTablet = screenWidth > 600;

Text('Screen width: $screenWidth')
```

► 2. LayoutBuilder

Explanation: Gives you the **constraints** (available width/height) of the parent widget — useful for building responsive widgets that adapt to their container, not just the full screen.

```

LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth > 600) {
      return const Text('Wide layout');
    } else {
      return const Text('Narrow layout');
    }
  },
)

```

► 3. Orientation

Explanation: Detects portrait vs. landscape, letting you rearrange layout (e.g. switch from `Column` to `Row`).

```

OrientationBuilder(
  builder: (context, orientation) {
    return orientation == Orientation.portrait
      ? const Text('Portrait mode')
      : const Text('Landscape mode');
  },
)

```

► 4. Responsive Design

Explanation: Combining `MediaQuery`, `LayoutBuilder`, `Expanded` / `Flexible`, and breakpoints lets one codebase adapt gracefully across phones, tablets, and desktop/web.

```

Widget build(BuildContext context) {
  final width = MediaQuery.of(context).size.width;
  int crossAxisCount = width > 900 ? 4 : (width > 600 ? 3 : 2);

  return GridView.count(
    crossAxisCount: crossAxisCount,
    children: List.generate(0, (i) => Card(child: Center(child: Text('Item $i')))),
  );
}

```

Animations

► 1. AnimatedContainer

Explanation: Automatically animates changes to a `Container`'s properties (color, size, padding) over a given duration — no manual `AnimationController` needed.

```

class ToggleBox extends StatefulWidget {
  const ToggleBox({super.key});
  @override
  State<ToggleBox> createState() => _ToggleBoxState();
}

class _ToggleBoxState extends State<ToggleBox> {
  bool expanded = false;

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () => setState(() => expanded = !expanded),
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 300),
        width: expanded ? 300 : 100,
        height: expanded ? 300 : 100,
        color: expanded ? Colors.blue : Colors.red,
      ),
    );
  }
}

```

► 2. Hero

Explanation: Creates a smooth shared-element transition of a widget between two screens (e.g. a thumbnail flying into a full detail image) as the user navigates.

```
// Screen A
Hero(
  tag: 'product-image',
  child: Image.network('https://picsum.photos/100'),
)

// Screen B — same tag, different size
Hero(
  tag: 'product-image',
  child: Image.network('https://picsum.photos/400'),
)
```

► 3. AnimationController

Explanation: The core low-level animation driver — controls the timeline (start, stop, reverse, repeat) of an animation, paired with a `TickerProvider` (usually via `SingleTickerProviderStateMixin`).

```
class SpinningIcon extends StatefulWidget {
  const SpinningIcon({super.key});
  @override
  State<SpinningIcon> createState() => _SpinningIconState();
}

class _SpinningIconState extends State<SpinningIcon> with SingleTickerProviderStateMixin {
  late AnimationController _controller;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(vsync: this, duration: const Duration(seconds: 2))..repeat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return RotationTransition(turns: _controller, child: const Icon(Icons.autorenew, size: 40));
  }
}
```

► 4. Tween Animation

Explanation: A `Tween` (short for “in-between”) defines a range of values (e.g. 0.0 → 1.0, or one `Color` to another) that an `AnimationController` interpolates through over time.

```
late Animation<double> _sizeAnimation;

@override
void initState() {
  super.initState();
  _controller = AnimationController(vsync: this, duration: const Duration(seconds: 1));
  _sizeAnimation = Tween<double>(begin: 50, end: 150).animate(_controller);
  _controller.forward();
}

// In build:
AnimatedBuilder(
  animation: _sizeAnimation,
  builder: (context, child) => Container(
    width: _sizeAnimation.value,
    height: _sizeAnimation.value,
    color: Colors.purple,
  ),
)
```

Device Features

► 1. Camera

Explanation: The `camera` (or simpler `image_picker`) package lets the app capture photos/video directly from the device camera.

```
dependencies:
  image_picker: ^1.1.2
```

```
import 'package:image_picker/image_picker.dart';

Future<void> takePhoto() async {
  final picker = ImagePicker();
  final photo = await picker.pickImage(source: ImageSource.camera);
  if (photo != null) print('Photo saved at: ${photo.path}');
}
```

► 2. Gallery

Explanation: Same `image_picker` package lets users pick existing photos from their device gallery instead of the camera.

```
Future<void> pickFromGallery() async {
  final picker = ImagePicker();
  final image = await picker.pickImage(source: ImageSource.gallery);
  if (image != null) print('Selected: ${image.path}');
}
```

► 3. GPS/Location

Explanation: The `geolocator` package retrieves the device's current GPS coordinates — useful for location-tagging, maps, or nearby-search features.

```
dependencies:
  geolocator: ^13.0.1
```

```
import 'package:geolocator/geolocator.dart';

Future<Position> getCurrentLocation() async {
  bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
  if (!serviceEnabled) throw Exception('Location services disabled');

  LocationPermission permission = await Geolocator.checkPermission();
  if (permission == LocationPermission.denied) {
    permission = await Geolocator.requestPermission();
  }

  return Geolocator.getCurrentPosition();
}
```

► 4. Permissions

Explanation: The `permission_handler` package provides a unified API to request/check runtime permissions (camera, location, storage, notifications) across Android/iOS.

```
dependencies:
  permission_handler: ^11.3.1
```

```
import 'package:permission_handler/permission_handler.dart';

Future<void> requestCameraPermission() async {
  var status = await Permission.camera.request();
  if (status.isGranted) {
    print('Camera permission granted');
  } else {
    print('Camera permission denied');
  }
}
```

► 5. Notifications

Explanation: `flutter_local_notifications` schedules and displays notifications directly from the app (independent from Firebase Cloud Messaging's push notifications).

```
dependencies:
  flutter_local_notifications: ^17.2.2
```

```
import 'package:flutter_local_notifications/flutter_local_notifications.dart';

final flutterLocalNotificationsPlugin = FlutterLocalNotificationsPlugin();

Future<void> showNotification() async {
  const androidDetails = AndroidNotificationDetails('channel_id', 'General', importance: Importance.high);
  const details = NotificationDetails(android: androidDetails);
  await flutterLocalNotificationsPlugin.show(0, 'Reminder', 'Thesis defense at 2PM', details);
}
```

► 6. File Picker

Explanation: The `file_picker` package lets the user select **any file type** (PDF, docx, zip) from device storage, not just images.

```
dependencies:
  file_picker: ^8.0.6
```

```
import 'package:file_picker/file_picker.dart';

Future<void> pickDocument() async {
  FilePickerResult? result = await FilePicker.platform.pickFiles(type: FileType.custom, allowedExtensions: ['pdf', 'docx']);
  if (result != null) {
    print('Picked file: ${result.files.single.path}');
  }
}
```

Architecture

► 1. MVC (Model-View-Controller)

Explanation: Separates an app into **Model** (data), **View** (UI), and **Controller** (logic connecting them). Simple to grasp, but in Flutter the “Controller” role often blurs into the widget itself for small apps.

```
// Model
class Task {
  String title;
  bool done;
  Task(this.title, {this.done = false});
}

// Controller
class TaskController {
  final List<Task> _tasks = [];
  List<Task> get tasks => _tasks;
  void addTask(String title) => _tasks.add(Task(title));
}

// View (widget) calls the controller and rebuilds itself
```

► 2. MVVM (Model-View-ViewModel)

Explanation: The **ViewModel** exposes observable state (often via `ChangeNotifier` /Provider/Riverpod) that the **View** listens to, without the View directly touching business logic — a natural fit for Flutter’s reactive widget model.

```
// ViewModel
class TaskViewModel extends ChangeNotifier {
  final List<Task> _tasks = [];
  List<Task> get tasks => List.unmodifiable(_tasks);

  void addTask(String title) {
    _tasks.add(Task(title));
    notifyListeners();
  }
}

// View
class TaskListView extends StatelessWidget {
  const TaskListView({super.key});
  @override
  Widget build(BuildContext context) {
    final vm = context.watch<TaskViewModel>();
    return ListView(children: vm.tasks.map((t) => ListTile(title: Text(t.title))).toList());
  }
}
```

► 3. Clean Architecture

Explanation: Organizes code in concentric layers — **Presentation** (UI) → **Domain** (business logic/use cases, framework-independent) → **Data** (repositories, API/DB implementations). Dependencies only point **inward**, making the domain layer testable and framework-agnostic.

```
lib/
├─ presentation/  # widgets, screens, view models
├─ domain/        # entities, use cases, repository interfaces
└─ data/          # repository implementations, API/DB code
```

```
// domain/entities/task.dart
class Task {
  final String id;
  final String title;
  Task(this.id, this.title);
}

// domain/repositories/task_repository.dart (interface)
abstract class TaskRepository {
  Future<List<Task>> getTasks();
}

// domain/usecases/get_tasks.dart
class GetTasks {
  final TaskRepository repository;
  GetTasks(this.repository);
  Future<List<Task>> call() => repository.getTasks();
}
```

► 4. Repository Pattern

Explanation: A **Repository** is an abstraction layer between business logic and data sources (API, local DB, cache) — the rest of the app doesn't need to know *where* data comes from, only that it can ask the repository for it.

```
abstract class TaskRepository {
  Future<List<Task>> getTasks();
}

class TaskRepositoryImpl implements TaskRepository {
  final http.Client client;
  TaskRepositoryImpl(this.client);

  @override
  Future<List<Task>> getTasks() async {
    final response = await client.get(Uri.parse('https://api.example.com/tasks'));
    final List data = jsonDecode(response.body);
    return data.map((json) => Task(json['id'], json['title'])).toList();
  }
}
```

Testing

► 1. Unit Test

Explanation: Tests a single function/class in isolation — no UI, no widgets. Fast and the foundation of a healthy test suite.

```
dev_dependencies:
  test: ^1.25.2
```

```
import 'package:test/test.dart';

int add(int a, int b) => a + b;

void main() {
  test('add() returns correct sum', () {
    expect(add(2, 3), equals(5));
  });
}
```

► 2. Widget Test

Explanation: Tests a single widget in a simulated environment (**WidgetTester**) — checks that UI renders correctly and responds to interaction, without needing a real device/emulator.


```
import 'package:flutter_test/flutter_test.dart';
import 'package:flutter/material.dart';

void main() {
  testWidgets('Counter increments on tap', (WidgetTester tester) async {
    await tester.pumpWidget(const MaterialApp(home: Counter()));

    expect(find.text('Count: 0'), findsOneWidget);

    await tester.tap(find.byType(ElevatedButton));
    await tester.pump(); // rebuild after setState

    expect(find.text('Count: 1'), findsOneWidget);
  });
}
```

► 3. Integration Test

Explanation: Tests the **full app** end-to-end on a real device/emulator — simulates real user flows across multiple screens (e.g. login → dashboard → logout).

```
dev_dependencies:
  integration_test:
    sdk: flutter
```

```
import 'package:integration_test/integration_test.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/main.dart' as app;

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('full login flow', (tester) async {
    app.main();
    await tester.pumpAndSettle();

    await tester.enterText(find.byKey(const Key('emailField')), 'test@test.com');
    await tester.enterText(find.byKey(const Key('passwordField')), '123456');
    await tester.tap(find.byKey(const Key('loginButton')));
    await tester.pumpAndSettle();

    expect(find.text('Welcome'), findsOneWidget);
  });
}
```

Deployment

► 1. Generate APK

Explanation: An APK (Android Package) is the installable Android build format — good for testing/sideload, or distributing outside Google Play.

```
flutter build apk --release
# output: build/app/outputs/flutter-apk/app-release.apk
```

► 2. Generate AAB

Explanation: An AAB (Android App Bundle) is Google Play's **required** publishing format — Play Store generates optimized APKs per device from it, reducing download size.

```
flutter build appbundle --release
# output: build/app/outputs/bundle/release/app-release.aab
```

► 3. Release Build

Explanation: A release build strips debug info, enables optimizations (tree-shaking, minification), and requires proper **app signing** (a keystore) before publishing.

```
# Generate a signing key (once)
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA -keysize 2048 -validity 10000 -alias upload

# Reference the key in android/key.properties, then:
flutter build appbundle --release
```

► 4. Publish to Google Play

Explanation: Once you have a signed AAB, upload it via the **Google Play Console** — fill in store listing (screenshots, description), set content rating, pricing, and submit for review.

Checklist:

1. Create a Google Play Developer account (one-time fee)
2. Create a new app in Play Console
3. Upload app-release.aab under "Production" (or "Internal testing" first)
4. Complete store listing, privacy policy, content rating
5. Submit for review

Git

► 1. Git Basics

Explanation: Git tracks changes to your code over time, letting you save checkpoints (commits) and revert if something breaks.

```
git init                # start tracking a project
git add .               # stage all changed files
git commit -m "Initial commit" # save a checkpoint
git status              # see what's changed
git log                 # view commit history
```

► 2. GitHub

Explanation: GitHub hosts Git repositories remotely — enables backup, collaboration, and portfolio visibility (like your `Dave640-creator` repos).

```
git remote add origin https://github.com/username/repo.git
git push -u origin main
git pull origin main # get latest changes from remote
```

► 3. Branches

Explanation: A branch is an isolated line of development — work on new features without affecting the stable `main` branch.

```
git branch feature/login # create a branch
git checkout feature/login # switch to it
# or in one step:
git checkout -b feature/login
```

► 4. Merge

Explanation: Combines changes from one branch into another (e.g. merging a finished feature branch back into `main`).

```
git checkout main
git merge feature/login
```

► 5. Pull Requests

Explanation: On GitHub, a Pull Request (PR) proposes merging your branch into another (usually `main`), enabling code review and discussion before changes are merged — critical for group capstone/thesis projects with multiple contributors like Trazona and Napone.

```
# After pushing your feature branch:
git push origin feature/login
# Then open a Pull Request on GitHub's web UI comparing feature/login → main
```

Projects (Practice)

Building small projects is how the concepts above actually stick. Work through these roughly in order — each one deliberately combines the skills from the phase you just studied.

► 1. Calculator

Focus: Layout widgets, `Row` / `Column` / `GridView`, functions, operators. A basic 4-function calculator UI with a `GridView` of number/operator buttons and a display `Text` at the top, driven by `setState`.

► 2. BMI Calculator

Focus: `TextField`, forms, if-else logic, basic math. Take height/weight input, compute BMI, and branch (if-else) to show a category (Underweight/Normal/Overweight).

► 3. To-Do App

Focus: `ListView.builder`, `StatefulWidget`, CRUD logic, local storage (`SharedPreferences` or `Hive`). Add/edit/delete/check-off tasks; persist them so the list survives app restarts.

► 4. Notes App

Focus: SQLite or Hive, navigation (list screen → detail/edit screen), CRUD. A more structured to-do — full notes with title + body, browsable list, and a detail/edit screen with `Navigator`.

► 5. Expense Tracker

Focus: Local database, `ListView`, charts (optional package like `fl_chart`), state management (Provider/Riverpod). Log expenses by category, show a running total, and visualize spending trends.

► 6. Weather App

Focus: `http` package, REST API consumption, JSON parsing, loading/error states. Call a public weather API (e.g. OpenWeatherMap), parse JSON into a `Weather` model, and display current conditions with icons.

► 7. QR Scanner

Focus: Camera/device features, third-party packages (e.g. `mobile_scanner`), permissions. Scan a QR code with the camera and display/act on the decoded content.

► 8. Music Player

Focus: State management, `Stream`s (playback position), local file handling, custom UI/animations. Play local or remote audio files (e.g. via `just_audio`), with play/pause/seek controls and animated album art — this maps directly onto your `smart_music_player` Flutter project.

► 9. Chat App

Focus: Firebase Authentication + Firestore (real-time `Stream`s), navigation, clean architecture. Real-time messaging between users backed by Firestore's live snapshots; a great capstone-level Firebase exercise.

► 10. E-commerce App

Focus: Full architecture (MVVM/Clean), state management, REST/Firebase backend, navigation, animations. Product catalog (`GridView`), product detail (`Hero` animation), cart (state management), and checkout flow.

► 11. Food Delivery App

Focus: Everything above plus GPS/location, real-time order tracking (Streams/Firestore), push notifications (FCM). Location-aware ordering with live order-status updates and delivery notifications.

► 12. Capstone Project

Focus: Everything combined — Clean Architecture / Repository Pattern, state management, Firebase or REST + SQLite backend, testing, and a signed release build published (or ready) for Google Play. This is where your own thesis/capstone system (e.g. the mobile-based Farm and Agricultural Forecasting System) comes together: proper architecture, tested business logic, real backend integration, and a polished, non-generic UI.

Final Notes

- Work top-to-bottom: Dart fundamentals → OOP → Advanced Dart → Flutter basics → widgets → navigation → state management → data/backend → polish (responsive UI, animation, device features) → architecture/testing/deployment.
- Type out every example yourself rather than copy-pasting — the goal is muscle memory, not just recognition.
- Once you're comfortable with `setState`, move to **Provider** or **Riverpod** early — most real Flutter codebases (and your own capstone/mobile projects) will expect proper state management rather than `setState` scattered everywhere.
- For your own projects (music player, capstone systems), prioritize: correct project structure → clean architecture layers → tested business logic → then UI polish.